

```
import streamlit as st
import anthropic
import base64
import json
import csv
import io
import os
import uuid
from datetime import datetime
from pathlib import Path
```

```
# — Page config —————
st.set_page_config(
    page_title="FlipFlow",
    page_icon="🔄",
    layout="wide",
    initial_sidebar_state="collapsed",
)
```

```
# — CSS —————
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Space+Mono:wght@400;700&fam
```

```
/* — Root tokens — */
:root {
    --bg:          #0d0f14;
    --surface:     #161922;
    --card:        #1e2330;
    --border:      #2a3040;
    --accent:      #00e5a0;
    --accent2:     #7c6af7;
    --warn:        #f7a24a;
    --danger:      #f75a5a;
    --text:        #e8ecf4;
    --muted:       #7a8499;
    --mono:        'Space Mono', monospace;
    --sans:        'DM Sans', sans-serif;
}
```

```
/* — Global reset — */
html, body, [class*="css"] {
    font-family: var(--sans) !important;
    background-color: var(--bg) !important;
```

```

    color: var(--text) !important;
}

/* — Hide Streamlit chrome — */
#MainMenu, footer, header { visibility: hidden; }
.block-container { padding-top: 1.5rem !important; max-width: 1280px; }

/* — App header — */
.ff-header {
    display: flex; align-items: center; gap: 14px;
    padding: 1.2rem 1.6rem;
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 14px;
    margin-bottom: 1.5rem;
}
.ff-logo { font-family: var(--mono); font-size: 1.55rem; font-weight: 700; color:
.ff-tagline { font-size: 0.82rem; color: var(--muted); margin-top: 2px; }

/* — Tab bar — */
.stTabs [data-baseweb="tab-list"] {
    background: var(--card) !important;
    border-radius: 10px !important;
    border: 1px solid var(--border) !important;
    padding: 4px !important;
    gap: 4px !important;
}
.stTabs [data-baseweb="tab"] {
    font-family: var(--sans) !important;
    font-weight: 600 !important;
    font-size: 0.88rem !important;
    color: var(--muted) !important;
    border-radius: 8px !important;
    padding: 8px 20px !important;
    border: none !important;
}
.stTabs [aria-selected="true"] {
    background: var(--accent) !important;
    color: #0d0f14 !important;
}
.stTabs [data-baseweb="tab-panel"] { padding-top: 1.2rem !important; }

/* — Cards — */
.ff-card {
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 12px;

```

```

padding: 1.2rem 1.4rem;
margin-bottom: 1rem;
}
.ff-card-accent { border-left: 3px solid var(--accent); }
.ff-card-warn    { border-left: 3px solid var(--warn); }
.ff-card-danger { border-left: 3px solid var(--danger); }

/* — Metric tiles — */
.ff-metrics { display: flex; gap: 12px; flex-wrap: wrap; margin-bottom: 1.2rem; }
.ff-metric {
  flex: 1; min-width: 130px;
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 14px 16px;
}
.ff-metric-label { font-size: 0.72rem; color: var(--muted); text-transform: upper }
.ff-metric-value { font-family: var(--mono); font-size: 1.45rem; font-weight: 700 }
.ff-metric-value.green { color: var(--accent); }
.ff-metric-value.purple { color: var(--accent2); }
.ff-metric-value.orange { color: var(--warn); }

/* — Result row — */
.ff-result {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 1rem 1.3rem;
  margin-bottom: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 8px;
}
.ff-result-title { font-weight: 600; font-size: 1rem; }
.ff-result-meta { display: flex; gap: 16px; flex-wrap: wrap; font-size: 0.82rem; }
.ff-verdict { font-family: var(--mono); font-size: 0.78rem; font-weight: 700; padding: 0 10px; }
.verdict-strong { background: rgba(0,229,160,0.15); color: var(--accent); border: 1px solid var(--border); }
.verdict-decent { background: rgba(124,106,247,0.15); color: var(--accent2); border: 1px solid var(--border); }
.verdict-low    { background: rgba(247,90,90,0.15); color: var(--danger); border: 1px solid var(--border); }

/* — Upload area — */
.stFileUploader > div { border-radius: 12px !important; }

/* — Buttons — */
.stButton > button {
  background: var(--accent) !important;
  color: #0d0f14 !important;
}

```

```

    font-family: var(--sans) !important;
    font-weight: 700 !important;
    font-size: 0.88rem !important;
    border: none !important;
    border-radius: 8px !important;
    padding: 10px 22px !important;
    transition: opacity 0.15s !important;
}

.stButton > button:hover { opacity: 0.85 !important; }

/* — Secondary button — */
.stButton.secondary > button {
    background: var(--surface) !important;
    color: var(--text) !important;
    border: 1px solid var(--border) !important;
}

/* — Inputs — */
.stTextInput > div > div > input,
.stNumberInput > div > div > input,
.stTextArea > div > div > textarea,
.stSelectbox > div > div {
    background: var(--surface) !important;
    border: 1px solid var(--border) !important;
    border-radius: 8px !important;
    color: var(--text) !important;
    font-family: var(--sans) !important;
}

/* — Divider — */
hr { border-color: var(--border) !important; }

/* — Progress / spinner — */
.stProgress > div > div { background: var(--accent) !important; }

/* — Table — */
.stDataFrame { border-radius: 10px !important; overflow: hidden; }

/* — Drop zone text — */
.upload-hint {
    background: var(--card);
    border: 2px dashed var(--border);
    border-radius: 12px;
    padding: 2rem;
    text-align: center;
    color: var(--muted);
    font-size: 0.92rem;
}

```

```

    margin-bottom: 1rem;
}
.upload-hint span { color: var(--accent); font-weight: 600; }

/* — Section label — */
.ff-section-label {
    font-family: var(--mono);
    font-size: 0.7rem;
    color: var(--muted);
    text-transform: uppercase;
    letter-spacing: 0.12em;
    margin-bottom: 8px;
}

/* — Thumb strip — */
.thumb-row { display: flex; gap: 6px; flex-wrap: wrap; margin-top: 4px; }
.thumb-img { width: 52px; height: 52px; object-fit: cover; border-radius: 6px; }
</style>
""", unsafe_allow_html=True)

# — Constants —————
DATA_FILE = "flipflow_data.csv"
MODEL      = "claude-sonnet-4-20250514"

# — Session state bootstrap —————
def _init():
    defaults = {
        "quick_results": [], # Phase 1 results
        "batch_items":   [], # Phase 2 / sell batch
        "api_key":       "",
    }
    for k, v in defaults.items():
        if k not in st.session_state:
            st.session_state[k] = v

_init()

# — Helpers —————
def img_to_b64(file_bytes: bytes) -> str:
    return base64.standard_b64encode(file_bytes).decode("utf-8")

def get_client():
    key = st.session_state.api_key.strip()
    if not key:
        st.error("⚠ Enter your Anthropic API key in the sidebar.")
        st.stop()
    return anthropic.Anthropic(api_key=key)

```

```
def verdict_html(verdict: str) -> str:
    v = verdict.lower()
    if "strong" in v:
        return f'<span class="ff-verdict verdict-strong">🚀 Strong</span>'
    elif "low" in v or "skip" in v:
        return f'<span class="ff-verdict verdict-low">⚠️ Low</span>'
    else:
        return f'<span class="ff-verdict verdict-decent">👍 Decent</span>'

# — Claude calls —————
def quick_analyze(image_bytes: bytes, planned_cost: float | None) -> dict:
    client = get_client()
    cost_hint = f"The user plans to pay ${planned_cost:.2f}." if planned_cost else ""
    prompt = f"""You are a resale expert analyzing a product screenshot for profit.
    {cost_hint}

Respond ONLY with a valid JSON object (no markdown, no extra text) with these examples:
{{
  "item_name": "short product name",
  "detected_price": 0.00,
  "est_resale_low": 0.00,
  "est_resale_high": 0.00,
  "recommended_sell_price": 0.00,
  "estimated_profit": 0.00,
  "margin_pct": 0,
  "verdict": "Strong | Decent | Low",
  "reasoning": "1-2 sentence explanation"
}}
```

```

Respond ONLY with a valid JSON object (no markdown, no extra text) with these examples:
{{
  "item_name": "short product name",
  "detected_price": 0.00,
  "est_resale_low": 0.00,
  "est_resale_high": 0.00,
  "recommended_sell_price": 0.00,
  "estimated_profit": 0.00,
  "margin_pct": 0,
  "verdict": "Strong | Decent | Low",
  "reasoning": "1-2 sentence explanation"
}}
```

Rules:

- detected_price: price visible in screenshot (0 if not visible)
- est_resale: realistic local FB Marketplace / OfferUp range
- recommended_sell_price: the HIGH end price that still sells fast
- estimated_profit: recommended_sell_price minus detected_price (or planned cost)
- margin_pct: integer 0-100
- verdict: "Strong" if margin >35%, "Decent" if 15-35%, "Low" if <15%

```

"""
    b64 = img_to_b64(image_bytes)
    resp = client.messages.create(
        model=MODEL,
        max_tokens=600,
        messages=[{
            "role": "user",
            "content": [
                {"type": "image", "source": {"type": "base64", "media_type": "image/jpeg", "data": b64}}
            ]
        }]
    )
    return resp.choices[0].message.content
```

```

        {"type": "text", "text": prompt}
    ]
    }]
)
raw = resp.content[0].text.strip()
# strip possible markdown fences
raw = raw.replace("```json", "").replace("```", "").strip()
return json.loads(raw)

def full_analyze(images_bytes: list[bytes], actual_cost: float, notes: str) -> dict:
    client = get_client()
    prompt = f"""You are a top-tier resale copywriter AND market analyst.

Actual purchase cost: ${actual_cost:.2f}
Seller notes: {notes or 'None'}

You have {len(images_bytes)} real photos of this item.

Respond ONLY with a valid JSON object (no markdown, no extra text):
{{
    "item_name": "clean product name",
    "condition": "Like New | Good | Fair | Poor",
    "fb_title": "optimized FB Marketplace title under 100 chars",
    "fb_description": "full FB Marketplace description, 3-4 paragraphs, highlight f
    "recommended_price": 0.00,
    "price_low": 0.00,
    "price_high": 0.00,
    "est_profit": 0.00,
    "margin_pct": 0,
    "market_score": 0,
    "reasoning": "2-3 sentence market analysis"
}}

Rules:
- fb_title: compelling, keyword-rich, no ALL CAPS
- recommended_price: sweet spot for fast sale at high margin
- market_score: 1-10 demand score
- est_profit: recommended_price minus actual_cost
- margin_pct: integer
"""
    content = []
    for b in images_bytes:
        content.append({"type": "image", "source": {"type": "base64", "media_type": "image", "data": b.decode("base64")}})
    content.append({"type": "text", "text": prompt})

    resp = client.messages.create(

```

```

        model=MODEL,
        max_tokens=1200,
        messages=[{"role": "user", "content": content}]
    )
    raw = resp.content[0].text.strip().replace("`json","").replace("`", "").st
    return json.loads(raw)

# — CSV persistence —————
def save_batch():
    if not st.session_state.batch_items:
        return
    fields = ["id", "item_name", "actual_cost", "suggested_price", "est_profit", "marg
              "status", "sell_price", "real_profit", "fb_title", "fb_description",
              "condition", "market_score", "notes", "created_at"]
    with open(DATA_FILE, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields, extrasaction="ignore")
        w.writeheader()
        w.writerows(st.session_state.batch_items)

def load_batch():
    if not os.path.exists(DATA_FILE):
        return
    with open(DATA_FILE, newline="") as f:
        reader = csv.DictReader(f)
        items = list(reader)
    if items:
        st.session_state.batch_items = items

def export_fb_csv() -> str:
    """Facebook Marketplace bulk upload format."""
    output = io.StringIO()
    fields = ["title", "description", "price", "condition", "category"]
    w = csv.DictWriter(output, fieldnames=fields)
    w.writeheader()
    for item in st.session_state.batch_items:
        w.writerow({
            "title":      item.get("fb_title") or item.get("item_name", ""),
            "description": item.get("fb_description", ""),
            "price":       item.get("suggested_price", ""),
            "condition":   item.get("condition", "Good"),
            "category":    "For Sale",
        })
    return output.getvalue()

# Load persisted data on first run
if "loaded" not in st.session_state:
    load_batch()

```



```

    st.session_state.loaded = True

# — Header —————
st.markdown("""
<div class="ff-header">
    <div>
        <div class="ff-logo">⚡ FlipFlow</div>
        <div class="ff-tagline">Screenshot → Analyze → List → Profit</div>
    </div>
</div>
""", unsafe_allow_html=True)

# — Sidebar: API Key —————
with st.sidebar:
    st.markdown("### 🔑 API Key")
    key_input = st.text_input(
        "Anthropic API Key",
        value=st.session_state.api_key,
        type="password",
        placeholder="sk-ant-...",
        help="Your key is stored only in this browser session – never saved to di",
        label_visibility="collapsed"
    )
    if key_input != st.session_state.api_key:
        st.session_state.api_key = key_input

    st.caption("🔒 Key lives in session memory only – never written to disk.")
    st.markdown("---")
    st.markdown("**Model:** `claude-sonnet-4-20250514`")
    st.markdown("**Vision:** ✅ Enabled")

# — Tabs —————
tab1, tab2 = st.tabs(["📷 Browse Mode – Quick Profit Check", "📦 Sell Batch"])

# =====
# PHASE 1 – Browse Mode
# =====

with tab1:
    st.markdown('<div class="ff-section-label">Phase 1 · Screenshot Analyzer</div>')

    with st.container():
        col_up, col_cost = st.columns([3, 1])
        with col_up:
            uploads = st.file_uploader(
                "Drop screenshots here (FB Marketplace, OfferUp, Amazon, store sh",
                accept_multiple_files=True,
                type=["png", "jpg", "jpeg", "webp"],

```

```

        key="phase1_uploads",
    )
    with col_cost:
        planned_cost = st.number_input(
            "My planned cost ($)",
            min_value=0.0, value=0.0, step=0.5,
            format="%.2f",
            help="Leave at 0 to auto-detect from screenshot",
        )

run_check = st.button("⚡ Quick Profit Check", disabled=not uploads)

if run_check and uploads:
    new_results = []
    bar = st.progress(0, text="Analyzing with Claude Vision...")
    for i, f in enumerate(uploads):
        bar.progress((i + 1) / len(uploads), text=f"Analyzing {f.name}...")
        cost_val = planned_cost if planned_cost > 0 else None
        try:
            data = quick_analyze(f.read(), cost_val)
            data["_filename"] = f.name
            data["_id"] = str(uuid.uuid4())
            new_results.append(data)
        except Exception as e:
            st.error(f"Error on {f.name}: {e}")
    bar.empty()
    # Prepend new results so latest are on top
    st.session_state.quick_results = new_results + st.session_state.quick_res

# — Summary dashboard —————
if st.session_state.quick_results:
    results = st.session_state.quick_results
    total_potential = sum(float(r.get("estimated_profit", 0) or 0) for r in results)
    total_cost = sum(float(r.get("detected_price", 0) or 0) for r in results)
    strong_count = sum(1 for r in results if "strong" in str(r.get("verdict", "")))
    avg_margin = sum(float(r.get("margin_pct", 0) or 0) for r in results)

    st.markdown(f"""
<div class="ff-metrics">
  <div class="ff-metric">
    <div class="ff-metric-label">Items Analyzed</div>
    <div class="ff-metric-value">{len(results)}</div>
  </div>
  <div class="ff-metric">
    <div class="ff-metric-label">Total Potential Profit</div>
    <div class="ff-metric-value green">${total_potential:,.2f}</div>
  </div>
    """)

```

```

<div class="ff-metric">
    <div class="ff-metric-label">Est. Total Buy Cost</div>
    <div class="ff-metric-value">${total_cost:,.2f}</div>
</div>
<div class="ff-metric">
    <div class="ff-metric-label">🚀 Strong Picks</div>
    <div class="ff-metric-value purple">{strong_count}</div>
</div>
<div class="ff-metric">
    <div class="ff-metric-label">Avg Margin</div>
    <div class="ff-metric-value orange">{avg_margin:.0f}%</div>
</div>
</div>
""" , unsafe_allow_html=True)

st.markdown("---")
st.markdown('<div class="ff-section-label">Results - click "I Bought This

for idx, r in enumerate(results):
    verdict = r.get("verdict", "Decent")
    card_cls = "ff-card-accent" if "strong" in verdict.lower() else ("ff-

with st.container():
    st.markdown(f'<div class="ff-card {card_cls}">', unsafe_allow_htm

c1, c2, c3, c4, c5, c6 = st.columns([3, 1.2, 1.2, 1.2, 1.2, 1.5])
with c1:
    st.markdown(f"***{r.get('item_name', 'Unknown Item')}***")
    st.caption(r.get("_filename", ""))
with c2:
    st.markdown(f"***Buy:** ` ${float(r.get('detected_price', 0)):.2

with c3:
    st.markdown(f"***Sell:** ` ${float(r.get('recommended_sell_pric

with c4:
    profit = float(r.get("estimated_profit", 0))
    color = "green" if profit > 0 else "red"
    st.markdown(f"***Profit:** :{color}[` ${profit:.2f}`]")
with c5:
    st.markdown(f"***Margin:** `{r.get('margin_pct', 0)}%`")
with c6:
    if st.button(f"🛒 I Bought This", key=f"buy_{r['_id']}"):
        st.session_state[f"show_buy_form_{r['_id']}"] = True

st.markdown(verdict_html(verdict), unsafe_allow_html=True)
st.caption(r.get("reasoning", ""))
st.markdown("</div>", unsafe_allow_html=True)

```



```

        "notes":          notes_input,
        "reasoning":      listing.get("reasoni
        "thumb_b64":      thumb_b64,
        "created_at":     datetime.now().strft
    }
    st.session_state.batch_items.append(batch
    save_batch()
    st.session_state[form_key] = False
    st.success(f"✅ Added '{batch_item['item_
    st.rerun()
except Exception as e:
    st.error(f"Analysis failed: {e}")

# Clear button
if st.button("🗑️ Clear Browse Results", key="clear_browse"):
    st.session_state.quick_results = []
    st.rerun()

else:
    st.markdown("""
    <div class="upload-hint">
        📷 Drop screenshots above to get started.<br>
        <span>FB Marketplace · OfferUp · Amazon · Thrift stores · Any product i
    </div>
    """, unsafe_allow_html=True)

# =====
# PHASE 2 — Sell Batch
# =====

with tab2:
    batch = st.session_state.batch_items

    # — Live dashboard —————
    total_spent    = sum(float(i.get("actual_cost",0) or 0) for i in batch)
    est_profit_all = sum(float(i.get("est_profit",0) or 0) for i in batch)
    realized       = sum(float(i.get("real_profit",0) or 0) for i in batch if i.g
    sold_count     = sum(1 for i in batch if i.get("status") == "Sold")
    margins        = [float(i.get("margin_pct",0) or 0) for i in batch if i.get("
    avg_m          = sum(margins)/len(margins) if margins else 0

    st.markdown(f"""
    <div class="ff-metrics">
        <div class="ff-metric">
            <div class="ff-metric-label">Items in Batch</div>
            <div class="ff-metric-value">{len(batch)}</div>
        </div>
        <div class="ff-metric">

```

```

        <div class="ff-metric-label">Total Spent</div>
        <div class="ff-metric-value orange">${total_spent:,.2f}</div>
    </div>
    <div class="ff-metric">
        <div class="ff-metric-label">Est. Profit</div>
        <div class="ff-metric-value green">${est_profit_all:,.2f}</div>
    </div>
    <div class="ff-metric">
        <div class="ff-metric-label">Realized Profit</div>
        <div class="ff-metric-value purple">${realized:,.2f}</div>
    </div>
    <div class="ff-metric">
        <div class="ff-metric-label">Sold</div>
        <div class="ff-metric-value">{sold_count}</div>
    </div>
    <div class="ff-metric">
        <div class="ff-metric-label">Avg Margin</div>
        <div class="ff-metric-value">{avg_m:.0f}%</div>
    </div>
</div>
""", unsafe_allow_html=True)

if batch:
    # Export button
    col_exp, _ = st.columns([1, 3])
    with col_exp:
        csv_data = export_fb_csv()
        st.download_button(
            "📄 Export CSV for FB Bulk Upload",
            data=csv_data,
            file_name=f"flipflow_fb_{datetime.now().strftime('%Y%m%d_%H%M')}.",
            mime="text/csv",
        )

    st.markdown("---")
    st.markdown('<div class="ff-section-label">Batch Items</div>', unsafe_all

    changed = False
    items_to_delete = []

    for idx, item in enumerate(batch):
        with st.container():
            st.markdown('<div class="ff-card">', unsafe_allow_html=True)

            # Row 1: thumb + item info
            col_thumb, col_info = st.columns([1, 7])
            with col_thumb:

```

```

        if item.get("thumb_b64"):
            st.markdown(
                f'", unsafe_allow_html=True)

# Process deletions and saves
if items_to_delete:
    for idx in sorted(items_to_delete, reverse=True):
        st.session_state.batch_items.pop(idx)
    save_batch()
    st.rerun()

if changed:
    save_batch()

else:
    st.markdown("""
<div class="upload-hint">
    📦 Your Sell Batch is empty.<br>
    <span>Go to Browse Mode → analyze a screenshot → click "I Bought This"<
</div>
""", unsafe_allow_html=True)

```